

Relazione Progetto MapReduce - libmr

Framework per analisi di file di testo

Maggio 2026

Contents

1	Introduzione	2
2	Interfaccia Pubblica	2
2.1	Macro di Configurazione	2
3	Architettura e Gestione Processi	3
3.1	Creazione dei Processi e Fork	3
3.2	Gestione dei File Descriptor e Pipeline	3
3.3	Coordinamento e Terminazione	4
4	Organizzazione dei Thread	4
4.1	Processo Mapper	4
4.2	Processo Reducer	4
5	Protocollo Interno e Sincronizzazione	5
5.1	Formato dei Messaggi	5
5.2	Sincronizzazione Interna	5
6	Strutture Dati e Formati	5
6.1	Tabella Hash nel Reducer	5
6.2	Formato di Output	6
7	Sistema di Logging	6
8	Test e Validazione	6
9	Conclusioni	7

1 Introduzione

Il progetto consiste nella realizzazione di un framework denominato `libmr`, scritto in linguaggio C, che implementa il paradigma MapReduce per l'analisi di file testuali su un singolo calcolatore. Il framework gestisce in modo autonomo la creazione di processi, la comunicazione tramite pipe e l'orchestrazione di thread C11, permettendo all'utente di concentrarsi esclusivamente sulla logica di *mapping* e *reducing*.

Il framework è ora completamente operativo e implementa tutte le funzionalità richieste, inclusa la pipeline multiprocesso e l'esecuzione multithreaded per la parallelizzazione del carico di lavoro.

2 Interfaccia Pubblica

L'interfaccia pubblica è definita in `include/mr.h` e si basa su un handle opaco `mr_t`. Le principali funzioni fornite sono:

- **mr_create**: Inizializza l'handle `mr_t` con le funzioni di mapper e reducer fornite dall'utente e gli attributi di configurazione.
- **mr_start**: Avvia l'elaborazione MapReduce in modo bloccante. Prende in input il percorso dei dati (file o directory) e il percorso del file di output.
- **mr_destroy**: Rilascia tutte le risorse associate all'elaborazione.
- **mr_attr_***: Funzioni per la gestione della struttura `mr_attr_t`, che permette di configurare il numero di thread worker (mapper e reducer), la dimensione delle code interne e il file di log.

2.1 Macro di Configurazione

Il file `include/config.h` definisce diverse macro che regolano il comportamento di default del framework:

- **MR_DEFAULT_MAPPER_THREADS**: Numero predefinito di thread worker per il processo Mapper (default: 4).
- **MR_DEFAULT_REDUCER_THREADS**: Numero predefinito di thread worker per il processo Reducer (default: 4).
- **MR_DEFAULT_QUEUE_SIZE**: Capacità massima predefinita delle code interne (default: 100).
- **MR_DEFAULT_LOG_FILE**: Nome predefinito del file di log ("`mr.log`").
- **MR_REDUCER_HASH_TABLE_SIZE**: Dimensione della tabella hash utilizzata nel Reducer per il raggruppamento (default: 1024).
- **MR_LOG_SEM_NAME**: Nome del semaforo POSIX utilizzato per la sincronizzazione del log tra processi (default: "`mr_log_sem`").
- **MR_LOG_ENABLE_TIMESTAMP**: Abilita o disabilita l'inserimento del timestamp in ogni riga di log (default: 1, abilitato).

- **MR_LOG_LEVEL:** Definisce il livello minimo di severità dei messaggi da registrare (DEBUG, INFO, WARNING, ERROR).

Le callback utente seguono le firme `mr_mapper_t` e `mr_reducer_t`, ricevendo dati in input e utilizzando funzioni di *emit* fornite dal framework per produrre risultati intermedi o finali.

3 Architettura e Gestione Processi

L'architettura di `libmr` si basa su una pipeline di tre processi distinti che comunicano in modo unidirezionale tramite pipe POSIX. La creazione dei processi avviene tramite `fork()` nel processo principale.

3.1 Creazione dei Processi e Fork

Il processo principale (*Main*) è responsabile del setup iniziale e del coordinamento. Il framework segue un modello di esecuzione basato su processi separati per garantire l'isolamento delle risorse e sfruttare il parallelismo a livello di sistema operativo.

La creazione della pipeline avviene in tre fasi principali:

1. **Setup delle Pipe:** Vengono create tre pipe anonime tramite la system call `pipe()`: `main_to_mapper`, `mapper_to_reducer` e `reducer_to_main`.
2. **Fork del Mapper:** Viene invocata la `fork()` per creare il processo Mapper. Il processo figlio:
 - Utilizza `dup2()` per collegare l'estremità di lettura di `main_to_mapper` allo standard input (`STDIN_FILENO`).
 - Utilizza `dup2()` per collegare l'estremità di scrittura di `mapper_to_reducer` allo standard output (`STDOUT_FILENO`).
 - Chiude tutti i descrittori di file delle pipe non necessari per evitare che i riferimenti rimasti aperti impediscano la corretta chiusura delle pipe stesse.
 - Esegue la funzione `mapper_process_main`.
3. **Fork del Reducer:** Una seconda `fork()` crea il processo Reducer. Il processo figlio:
 - Collega la lettura di `mapper_to_reducer` a `STDIN_FILENO`.
 - Collega la scrittura di `reducer_to_main` a `STDOUT_FILENO`.
 - Chiude i descrittori superflui.
 - Esegue la funzione `reducer_process_main`.

3.2 Gestione dei File Descriptor e Pipeline

Una gestione rigorosa dei descrittori di file è fondamentale per il funzionamento della pipeline. In ogni processo (Main, Mapper e Reducer), vengono chiuse sistematicamente tutte le estremità delle pipe che non vengono utilizzate direttamente.

Questo è essenziale per due motivi:

- **Propagazione dell'EOF:** Una pipe viene considerata chiusa in lettura solo quando tutti i descrittori di scrittura associati ad essa sono stati chiusi. Se un processo (ad esempio il Main) mantenesse aperta l'estremità di scrittura di una pipe che non usa, il processo destinatario resterebbe bloccato in una `read()` infinita, non ricevendo mai il segnale di fine file.
- **Prevenzione di Deadlock:** Senza la chiusura tempestiva dei descrittori, i processi potrebbero attendere indefinitamente dati che non arriveranno mai o segnali di terminazione che non possono essere generati.

3.3 Coordinamento e Terminazione

Il processo Main, dopo aver completato l'invio dei dati al Mapper, chiude la sua estremità di scrittura di `main_to_mapper`. Questa azione innesca una reazione a catena lungo la pipeline:

1. Il Mapper legge l'EOF, termina l'elaborazione, chiude il suo `STDOUT` (collegato a `mapper_to_reducer`) ed esce.
2. Il Reducer riceve l'EOF dal Mapper, conclude l'aggregazione, scrive i risultati finali sul suo `STDOUT` (collegato a `reducer_to_main`) ed esce.
3. Il Main legge i risultati finali, chiude il suo `STDIN` (collegato a `reducer_to_main`) e utilizza `waitpid()` per raccogliere gli stati di terminazione dei figli, garantendo che non rimangano processi zombie nel sistema.

4 Organizzazione dei Thread

Sia il Mapper che il Reducer sono multithreaded e utilizzano lo standard C11 (`threads.h`).

4.1 Processo Mapper

- **Thread Controller (Lettore):** Legge le righe serializzate dallo standard input e le inserisce in una coda sincronizzata.
- **Thread Worker Mapper:** Estraggono le righe dalla coda, invocano la funzione `mapper` dell'utente e inviano le coppie risultanti allo standard output. L'accesso allo standard output è protetto da un mutex per evitare l'interlacciamento dei messaggi.

4.2 Processo Reducer

- **Thread Controller (Lettore):** Legge le coppie dallo standard input e le raggruppa in una tabella hash.
- **Raggruppamento:** Una volta ricevuto l'EOF, il controller inserisce i nodi della tabella hash (contenenti token e lista di valori) in una coda per i worker.
- **Thread Worker Reducer:** Estraggono i gruppi dalla coda, applicano la funzione `reducer` dell'utente e inviano i risultati finali allo standard output (collegato al processo principale).

5 Protocollo Interno e Sincronizzazione

La comunicazione sulle pipe utilizza un protocollo binario basato su header a dimensione fissa (lunghezze esplicite).

5.1 Formato dei Messaggi

La comunicazione tra i processi avviene serializzando i dati attraverso messaggi binari strutturati. Ogni messaggio è composto da una struttura di header a dimensione fissa seguita da un payload a dimensione variabile:

- **Main → Mapper (Input):** Il trasferimento dei file da elaborare prevede due tipologie di messaggi. All'inizio di un nuovo file viene inviato un header `mr_file_name_header_t` (contenente il campo `file_name_length` di tipo `size_t`), seguito dai byte del percorso del file (senza terminatore `\0`). Successivamente, per le righe, viene inviato un header `mr_line_header_t` (contenente un flag booleano `eof` per indicare la fine del file precedente, e la `line_length` di tipo `size_t`), seguito dai byte della riga (esclusi `\n` e `\0`).
- **Mapper → Reducer (Intermedio):** Il trasferimento delle coppie intermedie utilizza l'header `mr_pair_header_t`, che specifica `token_length` e `value_length` (entrambi `size_t`). Questo header è immediatamente seguito dai byte del token e successivamente dai byte del valore opaco.
- **Reducer → Main (Output):** L'invio dei risultati aggregati avviene tramite un header `mr_result_header_t`, contenente `token_length` e `result_size` (entrambi `size_t`). Segue il payload con i byte del token e i byte del risultato.

Le funzioni `read_n` e `write_n` gestiscono letture e scritture parziali, garantendo l'integrità dei messaggi.

5.2 Sincronizzazione Interna

La sincronizzazione tra thread è gestita tramite:

- `mtx_t`: Mutex C11 per proteggere le code e l'accesso alle pipe di scrittura.
- `cnd_t`: Condition variables C11 per implementare il modello produttore-consumatore bloccante (attesa su coda piena/vuota).

6 Strutture Dati e Formati

6.1 Tabella Hash nel Reducer

Il raggruppamento per token avviene tramite una tabella hash con *chaining* (default dimensione 1024). Ogni nodo della tabella mantiene un array dinamico di strutture `mr_value_t` per raccogliere tutti i valori intermedi associati a un singolo token.

6.2 Formato di Output

Il file di output finale è un file binario a record. Ogni record contiene:

1. Lunghezza del token (size_t)
2. Byte del token (senza terminatore nullo)
3. Lunghezza del risultato (size_t)
4. Byte del risultato (opachi)

7 Sistema di Logging

Il logging è sincronizzato tramite semafori POSIX per permettere a più processi di scrivere atomicamente sullo stesso file. Il formato è:

```
<timestamp> <pname>[pid] <tname> <level>: <message>
```

Vengono registrati eventi critici come la creazione di pipe, fork, avvio/terminazione di thread e statistiche finali (righe lette, coppie prodotte, token distinti).

8 Test e Validazione

La validazione del framework è effettuata mediante il framework di test unitari **Unity**. I test coprono:

- **Core:** Verifica integralmente il ciclo di vita di un job MapReduce, dalla creazione alla distruzione, assicurando la corretta propagazione degli errori.
- **Mapper:** Verifica che la funzione di mapping produca le coppie corrette per input di test.
- **Reducer:** Verifica che la funzione di reducing produca i risultati attesi per gruppi di token e valori di test.
- **Logging:** Verifica che i messaggi di log siano formattati correttamente e che la sincronizzazione funzioni senza interlacciamento.
- **MapReduce (Interfaccia Pubblica):** Verifica la corretta creazione e distruzione dell'handle `mr_t`, la gestione degli attributi e la corretta propagazione degli errori.
- **Protocollo di Comunicazione:** Test che verificano la corretta serializzazione e deserializzazione dei messaggi tra i processi, assicurando che i dati inviati corrispondano a quelli ricevuti.
- **Queue:** Test che verificano il corretto funzionamento delle code sincronizzate, inclusi i casi di coda piena e vuota, e la corretta gestione della chiusura.

9 Conclusioni

`libmr` implementa con successo il modello MapReduce sfruttando le primitive POSIX e C11. L'architettura a pipeline garantisce un isolamento efficace tra le fasi di elaborazione, mentre il multithreading permette di scalare su sistemi multicore, rispettando appieno i requisiti di robustezza e determinismo richiesti.